TÜRKİYE CUMHURİYETİ
YILDIZ TEKNİK ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

POSTGRES VERİTABAN YÖNETİCİSİ

23011935 — Mohamed Abdellah BOU
21011933 — Zoaber Osama ALKATMAH

BILGISAYAR PROJESİ

Danışman
Doç.Dr. Oğuz ALTUN

Mart, 2026

TEŞEKKÜR

We would like to express our gratitude to our supervisor for this project Doç.Dr. Oğuz ALTUN for giving us the opportunity to work under his guidance on this project and their valuable insights and advices.

Special thanks to the faculty of the Computer engineering department at Yildiz Technical University for providing the knowledge and giving us the opportunity to conduct our computer project.

WE owe a debt of gratitude to our parents for their sacrifices and encouragement over the past few years.

Mohamed Abdellah BOU
Zoaber Osama ALKATMAH

İÇİNDEKİLER

KISALTMA LİSTESİ

AI	artificial intelligence
CSS	Cascading Style Sheets
HTML	Hypertext Markup Language
IDE	Integrated Development Environment
ORM	Object-Relational Mapping
pnpm	Performant Node Package Manager
SQL	Structured Query Language
UML	Unified Modeling Language

ŞEKİL LİSTESİ

ÖZET

POSTGRES VERİTABAN YÖNETİCİSİ

Mohamed Abdellah BOU
Zoaber Osama ALKATMAH

Bilgisayar Mühendisliği Bölümü
Bilgisayar Projesi

Danışman: Doç.Dr. Oğuz ALTUN

Will be added in the final submission

Anahtar Kelimeler:

ABSTRACT

NAME OF THE PROJECT

Mohamed Abdellah BOU
Zoaber Osama ALKATMAH

Department of Computer Engineering
Computer Project

Advisor: Doç.Dr. Oğuz ALTUN

Will be added in the final submission

Keywords:

1
Introduction

With exponential growth of data in our today's life because of the use of modern technologies such as internet relational data bases has been an indispensable thing, instead of having to deal with heavy desktop applications that requires local installation or web applications that rely on host servers in this project our aim is to design and create web interface for users to connect and manage their databases like PostgreSQL from the web browser instead of downloading heavy databases and having to deal with setup problems, this web application will also provide many other features such as a new creative way of viewing rows of data such as Card Views. It will also support high-dimensional vector embeddings. Users will also be able to view the log history of all the actions done on the provided databases the technologies like edge computing and serverless functions that is will be used in this project aim to provide a faster and more reliable and highly secure database tool that can be used from any device.

To better understand the foundation and the work done in this project the skills,knowledge and prerequisites needed for this project is good understanding frontend development using react and how the front end communicates with the backend using Apis , relational databases and SQL concepts like SQL commands, serverless and edge computing and finally Object-Relational Mappers (ORMs) like Prisma.

1.1 Objective

Our primary objective and motivation of this project is to develop and design a highly responsive and light weight database web application instead of relying on desktop bounded applications. It will provide an isolated workspace where users can authenticate, configure their database credentials, and manage connections without effecting security. it will also feature a built-in SQL editor that will process and interpret SQL queries on the go. In Addition to having some creative new

features on viewing data records by introducing dynamic data visualization, users can easily change between traditional table grids and modern, visual 'Card Views' to better analyze media-rich records. Besides, to keep with the modern AI-driven applications, this tool supports high-dimensional vector embeddings. It includes a dedicated semantic search interface where users can adjust mathematical similarity thresholds to find and rank conceptually related records. The web based app is going to utilizes some modern technologies like serverless functions and edge computing, the databases will be managed by users anywhere and from any device, serverless computing will also allow users to run the application without the need for them to manage hardware and virtual servers that run 24/7, automatically scaling the servers based on demand. Edge computing will also provide a fast, smooth experience by deploying serverless functions all over the world so users can have an instantaneous response regardless of their location meaning it will be geographically closer to the user.

1.2 Preliminary Study

Before the development of the database management system a preliminary research was conducted. Our aim was to analyze the current similar database management systems that are available. Usually, interacting with relational databases requires developers to rely on two main categories of software one is heavy desktop clients and server-hosted web applications. Some of the Desktop based applications like DBeaver and DataGrip offer advanced features but requires local installation on the user's machine and complex network configurations. Thus it is not portable and the user can not access the database from different devices. on the other hand there are some web based applications like phpMyAdmin, while they solve the portability issue they introduce other disadvantages such as requiring the user to purchase a dedicated host server that requires extra effort to maintain the server 24/7 . This might cause security issues and extra costs on the users. And they might be slower than the serverless functions that are scattered in multiple locations by the edge computing technology to deliver a faster responses. Also both of these categories do not offer the Vector records feature that is the new creative way of viewing rows of data that is going to be designed in the web application.

The analysis of the existing systems reveals a problem that we offer a solution to which is the need for a database tool that combines the portability of web applications and the benefits offered by the serverless workers.

2
SYSTEM ANALYSIS AND FEASIBILITY

2.1 System Analysis

2.1. System Analysis

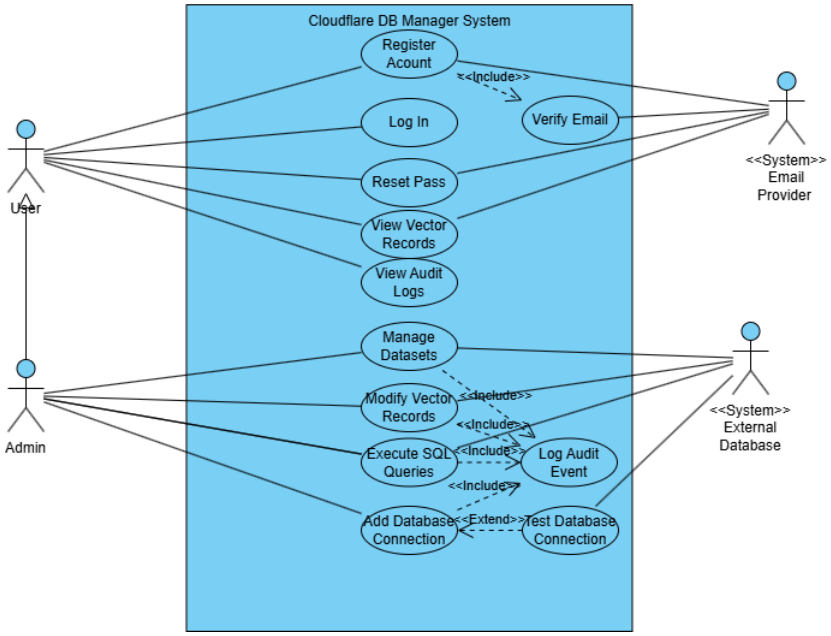
The final product will be evaluated according to some performance metrics these metrics will be analyzed in the system analysis section. Firstly, the main purpose of the proposed web based database manager is to be a bridge or act as an intermediary between the end-user and their relational databases. This analysis will ensure a suitable architectural solution, it will provide the structure, the key elements and functions of the application so we can test whether the needs are met when the project is completed. Here is some performance metrics.

1. **Query Executing Correctness:** The SQL editor should execute the queries from the user without any data loss or corruption, data must be modified exactly as the user intended
2. **Stability:** The app connection to the users external remote databases should be stable, if the network cuts out or freeze the error will be handled gracefully instead of crashing the users session.
3. **Handling The Complex Data Types:** The app will support some modern AI features it will be able to render complex data types and high dimension vector arrays.
4. **System Speed and Latency:** As it has been stated in the previous sections because of the app running on Cloudflare edge network API requests from the user will have a very fast responds with minimal latency.

2.2 Requirement Analysis

The project requirements was determined by a mix of some researches of existing database managers some meetings with the project instructor. In conclusion, the main functions in the application were defined.

The object-oriented approach was chosen for this project. Starting with the use case diagram that will define the flow of operations and actions by different users within the system.



Şekil 2.1 User Case diagram

Based on the use case diagram here is the main functional areas:

1. Authentication: First the user is met with the login page a user can login to an existing account or create a new one. This will ensure that the data is secure and safe.

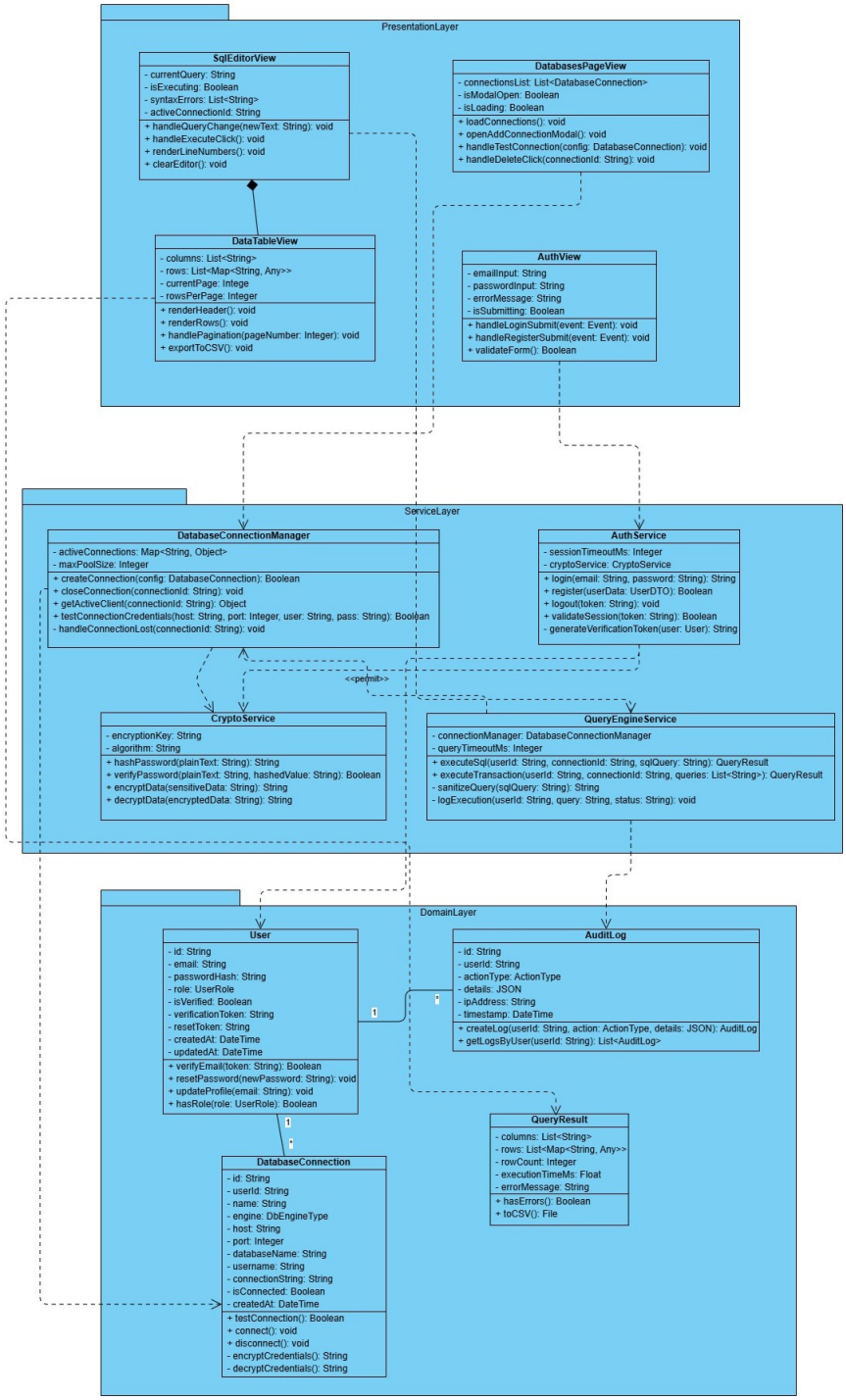
2. Manage Connections: User can view all saved databases and provide the credentials of a new one, the user will be able to ensure the connection with a click of a button.

3. Browse Schema and Tables: The users will be able to view the existing schemas and rows of tables they will be able to create, update delete rows of data.

4. Dynamic Views and Vector Search: User will be able to switch between cards view and vector arrays, users can launch a semantic search to find and rank conceptually similar records based on mathematical thresholds.

5. Execute SQL Query: The builtin SQL editor will execute the users SQL commands

Moving to the UML diagram :



Şekil 2.2 UML diagram

In this three tier architecture the data flows from the top to the bottom. All actions in the first layer doesn't interact with the database directly . First comes the presentation layer this layer represent the React interface that the user is going to interact with

it includes some actions like viewing the tables, triggering SQL queries, viewing the history log. Then we have the logic tier that perform all the tasks running on Cloudflare Workers that comes from the first layer it is like the brain that handles security, processes rules, authorizing users and much more. Finally the third layer the data access tier that handles the storage. After the request comes from first layer and gets verified by the second layer this layer executes the SQL commands and fetches data from the database.

2.3 Feasibility

2.3.1 Technical Feasibility

The technologies and softwares required to fully and successfully design and implement this project will be evaluated in the technical feasibility study. The following development tools and applications were chosen for the advantages they offer.

2.3.1.1 Software Feasibility

The project will implement RedwoodSDK framework the main language in this project is Typescript, which was used for both the front and the backend instead of JavaScript for the extra advantages it offers. Standard HTML and tailwind CSS are also used.

The user interface is built using React, which makes it easy to handle parts of the app dynamically (such as interactive data grids and SQL editors).The development environment relies on **Node.js** and **pnpm** for fast package management. Dev tools like git and and GitHub are utilized for codebase tracking and version control. and lastly the IDE used is vs code.

Cloudflare Workers were used instead of traditional servers like nodejs. Cloudflare Workers alows requests from the users to run on a global scale ensuring minimal latency

Database Management :**Prisma ORM** (Object-Relational Mapper) had been employed in this project to securely and easily connect to user databases. Prisma provides a very secure, type-safe query builder that acts as the bridge between the Cloudflare Worker backend and the external databases.

2.3.1.2 Hardware Feasibility

because this project is web based there was no need for any physical servers ,it was hosted in the cloud ,end Users will not need any special hardware or high-performance computers to use this application. Any device that has a search engine can be used.

The entire coding, testing, and deployment process of the project is being carried out on a personal laptop. Here are the specifications of the development pc:

Processor: 13th Gen Intel(R) Core(TM) i7-8565U Operating System: Linux RAM: 16.0 GB

These specifications are more than enough to run all the necessary development tools like Visual Studio Code, Node.js, and Vite.

2.3.2 Legal Feasibility

All resources used in this project are open sources so there is no violations. All processes are carried out legally and personal data is protected.

2.3.3 Economic Feasibility

The economic feasibility is broken down into three categories:

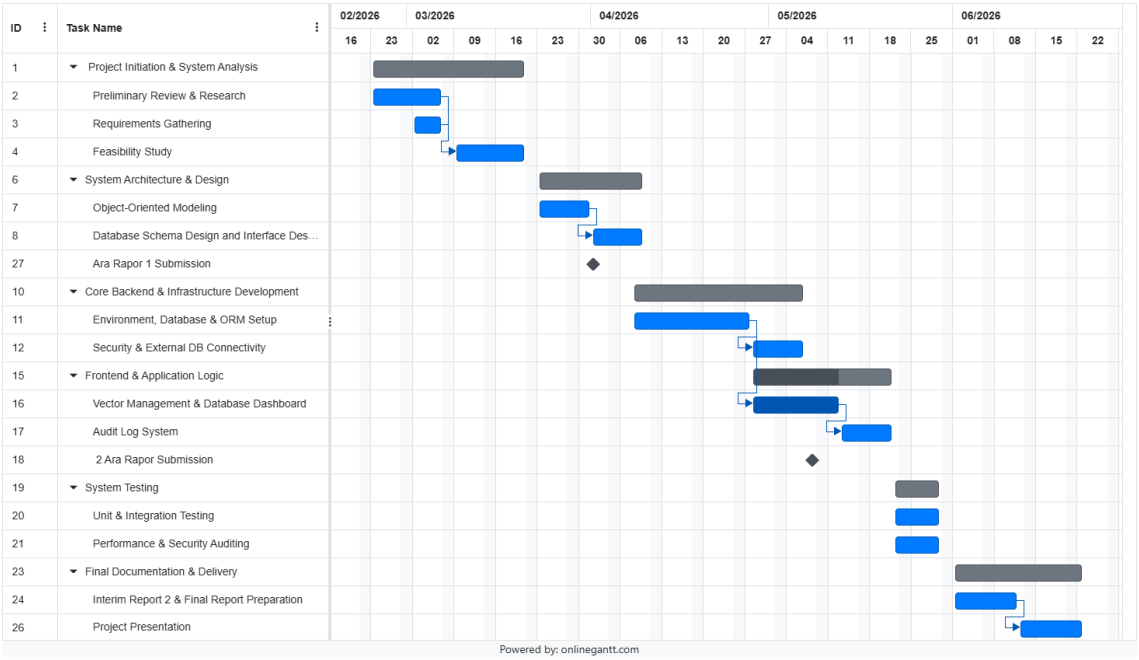
- **Labor Costs:** A team of two students is developing this as a graduation project so no labor costs was included.
- **Hosting and Hardware Costs:** By employing the free tiers of Cloudflare Workers and Cloudflare Pages for deployment the cost of hosting the web application and running the serverless backend is zero during the development and initial launch phases.
- **Software Costs:** Every development tool used (Visual Studio Code, Node.js, Git, pnpm) is completely free.

overall the whole project will almost not cost anything.

2.3.4 Labor and Time Feasibility

Only two students worked on this project. All the work done from fully developing the whole project to finishing and submitting the project report was divided over a

period of approximately 4 months the development life-cycle has been planned from February 2026 through June 2026.



Şekil 2.3 Gantt diagram

- **Weeks 1-2**

  Project Initiation , Feasibility study and System Analysis

- **Weeks 3-5**

  System Architecture And Design, drawing the diagrams and the first submission of the report.

- **Weeks 6-12**

  This phase involves building both the frontend and the backend, from designing the whole interface that the user is going to interact with to setting up the serverless environment, configuring the database ORM (Prisma), and establishing secure connectivity to external databases and lastly submitting the second report.

- **Weeks 13-14**

  System Testing, bugs and errors fixing and performance auditing.

- **Weeks 15-17**

  Final report preparation and documentation and Project presentation.

3
SYSTEM DESIGN

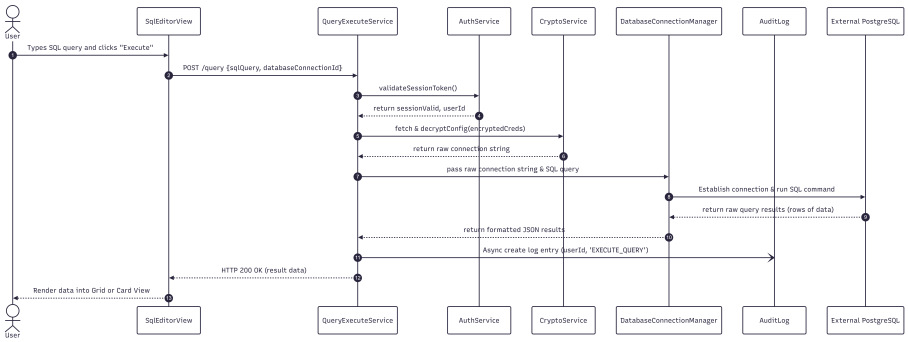
SYSTEM DESIGN

Following the requirement analysis and feasibility study, this section outlines the system design for the Postgres Database Manager. Since this is a software design project, the architecture is modeled using an object-oriented approach. This section is divided into software design, interface design, and database design.

3.1 Software Design

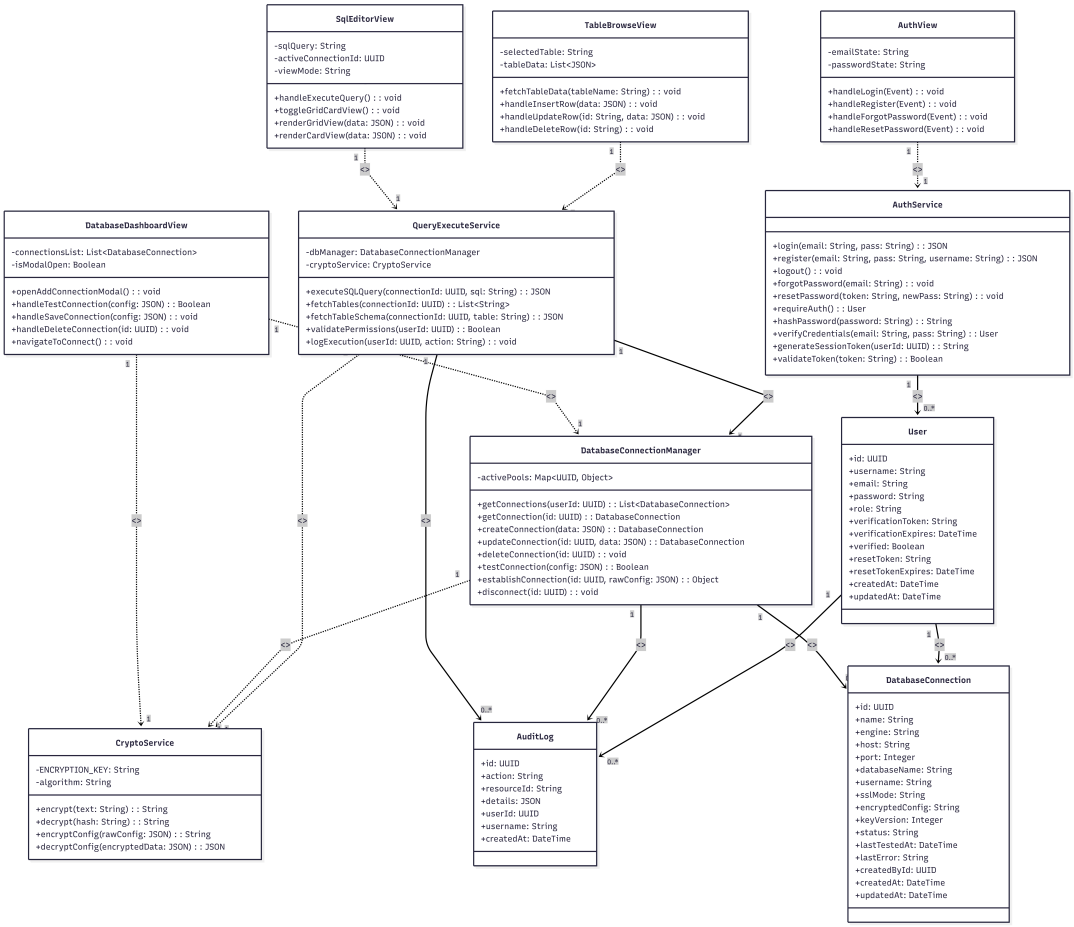
The software architecture is built upon a modern, three-tier object-oriented structure utilizing React (TypeScript) for the presentation layer, Cloudflare Workers for the serverless logic layer, and Prisma ORM for the data access layer. Because an object-oriented approach is preferred, the design is structured around distinct classes and services that communicate through specific interaction flows.

The interaction between system components is best illustrated using a sequence diagram. Figure 3.1 demonstrates the sequential flow of a user executing a SQL query. The process begins at the presentation layer (*SqlEditorView*), passes through the validation and authentication middleware in the logic layer (*QueryExecuteService*), and finally interacts with the external database via the *DatabaseConnectionManager*.



Şekil 3.1 Sequence diagram illustrating the SQL query execution flow.

In addition to the interaction diagrams, the structural design of the software classes has been refined. The design class diagram (extending the initial conceptual models) defines the specific attributes and methods for authentication (*AuthService*), connection handling (*DatabaseConnectionManager*), and cryptographic operations (*CryptoService*) .



Şekil 3.2 Detailed Design Class Diagram illustrating the presentation views, backend services, and database models.

3.2 Interface Design

The user interface (UI) design focuses on delivering a responsive, intuitive, and lightweight web environment for database management. To meet the core system interaction requirements, the application's screens are meticulously designed to seamlessly handle data inputs (*veri girişleri*), processing controls (*işlem kontrolleri*), and result outputs (*sonuç çıktıları*).

The interface is divided into four primary operational areas:

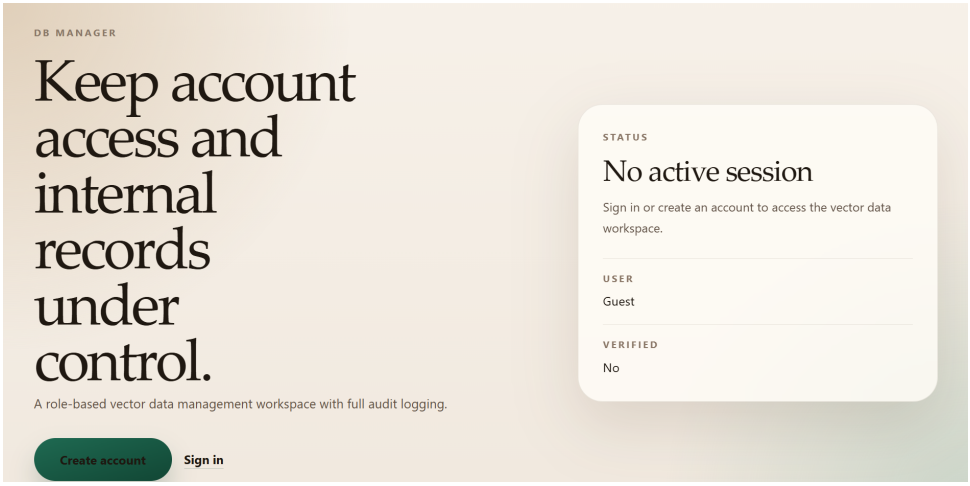
- **Authentication Interface:** Operating as the initial data input environment,

this screen securely captures user credentials (email and password) through web-based forms for registration and login. It utilizes strict form validation to ensure secure access to the application.

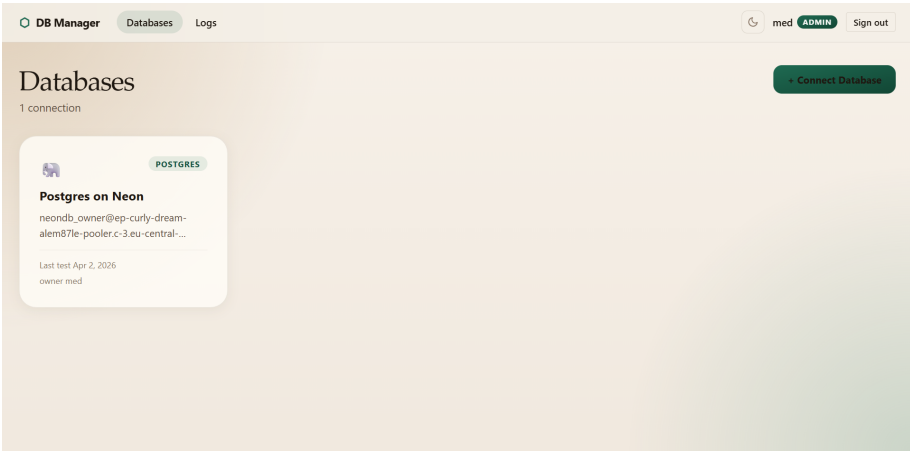
- **Database Dashboard (Connection Manager):** This screen acts as the central hub for managing active and saved databases. It utilizes a secure modal for the data input of PostgreSQL credentials (host, port, username, database name). Furthermore, it integrates essential processing controls, such as the "Test Connection" button, allowing users to validate their credentials prior to saving them.

- **SQL Editor:** A dedicated code-editor workspace that serves as the primary data input environment for writing raw SQL query strings. It features seamless processing controls, most notably the "Execute Query" button, which directly initiates the backend worker processes to run the user's commands.

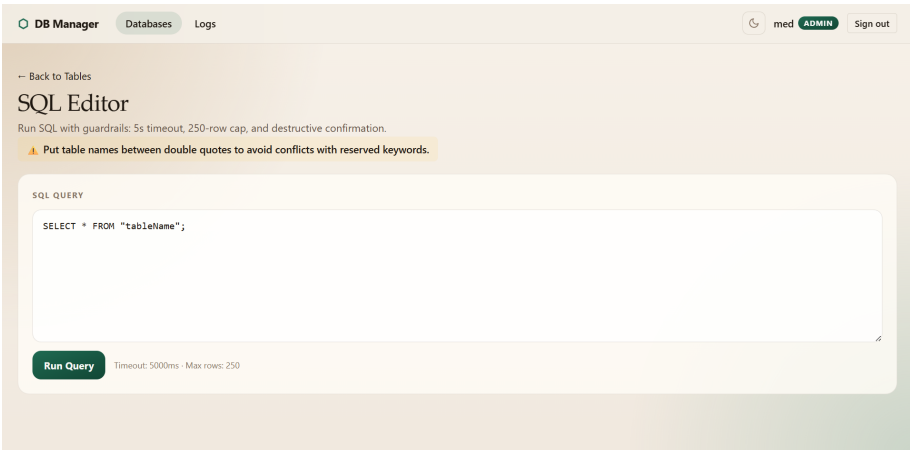
- **Table Browse & Result Outputs:** This interface is dedicated to displaying the system's dynamic result outputs. Query results are rendered in a highly flexible UI, allowing users to smoothly toggle between a traditional tabular "Grid View" and a modern "Card View" tailored for visualizing complex records. Additionally, this output-focused section encompasses the Audit Log Viewer, which presents a chronological report of all user actions and system activities.



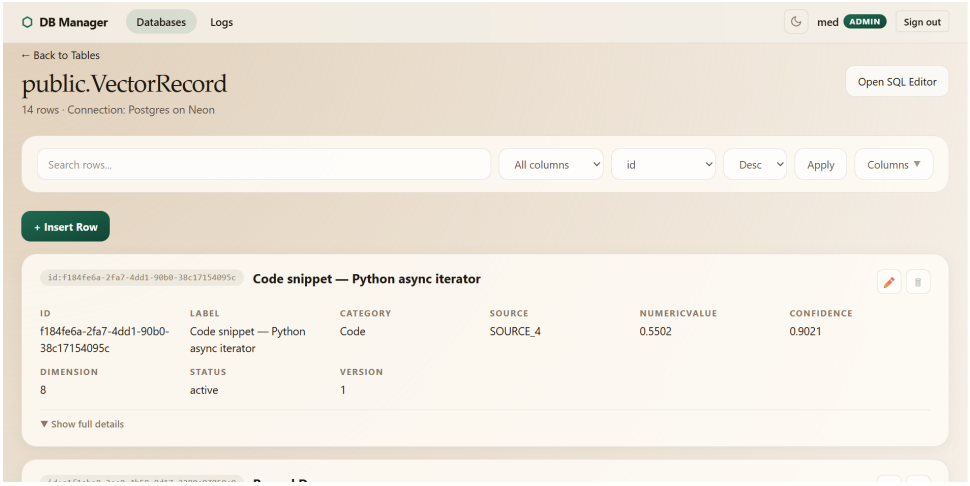
Şekil 3.3 Data Inputs: The Authentication interface securely capturing user credentials.



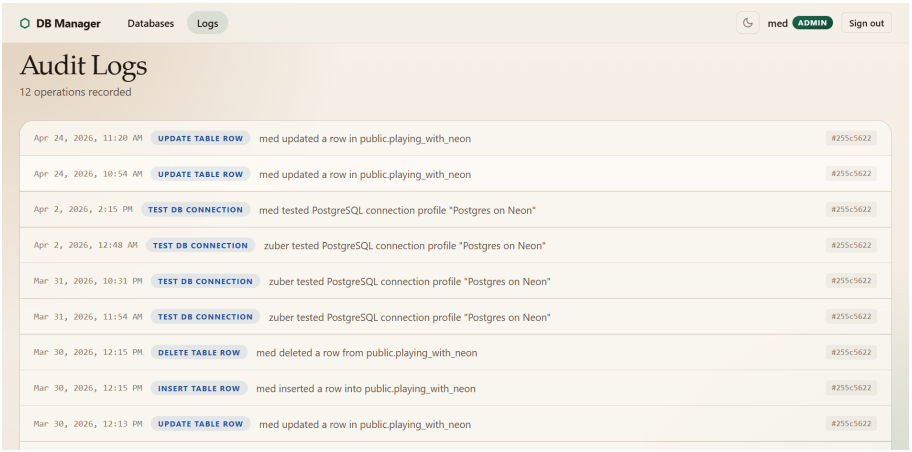
Şekil 3.4 Interface design for the Database Connection Dashboard.



Şekil 3.5 Interface design for the interactive SQL Editor and results viewer.



Şekil 3.6 Result Outputs: The Table Browse interface displaying dynamic data representations.

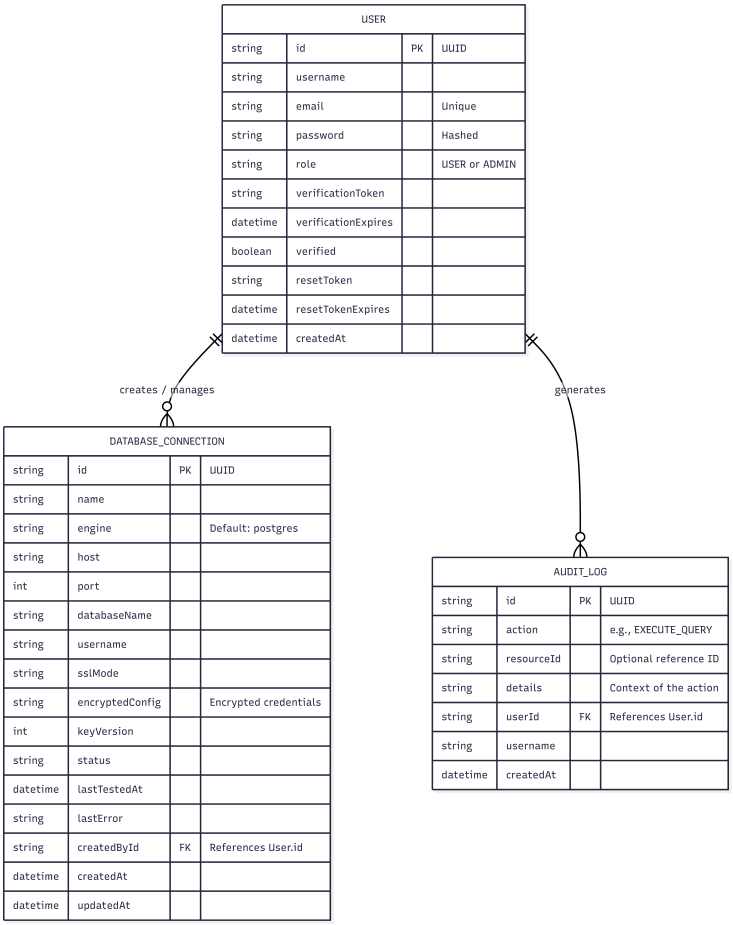


Şekil 3.7 Result Output: The Audit Log Viewer showing chronological user actions.

3.3 Database Design

The internal database of the application is designed to securely store user credentials, encrypted database connection strings, and audit logs. To clearly illustrate the entities that require data storage and the relationships between them, an Entity-Relationship (E-R) diagram has been utilized . The internal database schema relies on three primary entities defined via Prisma ORM:

- **User:** Stores user authentication details, including unique identifiers, hashed passwords, verification tokens, and role assignments (*USER* or *ADMIN*).
- **DatabaseConnection:** Stores the configurations for external PostgreSQL databases. To maintain maximum security, sensitive fields such as passwords and host URIs are processed through the *CryptoService* and stored in the *encryptedConfig* field. This entity maintains a Many-to-One relationship with the User entity.
- **AuditLog:** A tracking entity that records all significant actions performed by users, providing accountability and historical tracking for executed queries and connection modifications.



Şekil 3.8 Entity-Relationship (E-R) diagram of the internal system database.

While the application connects to external client databases, the internal state management strictly adheres to this E-R model. Because the application utilizes Prisma as a serverless Edge ORM, procedural data operations (which traditionally might rely on database-level stored procedures or triggers are instead securely managed and automatically triggered at the logic layer via Cloudflare Workers.

Stored Procedures and Triggers (Yordamlar ve Tetikleyiciler): According to the project guidelines, database design should account for database-level procedures and automatic triggers where necessary. However, because this system utilizes Prisma ORM operating within a globally distributed Cloudflare Edge network, traditional database-level triggers and stored procedures are bypassed in favor of application-level middleware. Automatic operations—such as cascading deletions or generating an *AuditLog* entry immediately after a user executes a query—are securely handled as programmatic "triggers" within the serverless logic tier, ensuring broader compatibility and edge-network performance.

Özgeçmiş

BİRİNCİ ÜYE

İsim-Soyisim: Mohamed Abdellah BOU
Doğum Tarihi ve Yeri: 16.07.2003, Magalahjar
E-mail: mohamed.bou@std.yildiz.edu.tr
Telefon: 0 552 650 56 18
Staj Tecrübeleri:

İKİNCİ ÜYE

İsim-Soyisim: Zoaber Osama ALKATMAH
Doğum Tarihi ve Yeri: 15.01.2004, RIYADH
E-mail: zoaber.alkatmah@std.yildiz.edu.tr
Telefon: 05421951204
Staj Tecrübeleri:

Proje Sistem Bilgileri

Sistem ve Yazılım: Linux İşletim Sistemi, TypeScript
Gerekli RAM: 2GB
Gerekli Disk: 256MB